

מטלה 4- הנדסת תוכנה

רעות בכור: 323834309 , לידור אייהוני: 209092287, רונן צ'רשניה: 207304338

חלק A:

תרחיש בדיקה 1: קביעת אימון על ידי מאמן והופעתו אצל המתאמן

שם התרחיש:

קביעת אימון על ידי מאמן והצגתו אצל מתאמן שנבחר

תנאי קדם:

1. האפליקציה מותקנת ופועלת.
2. קיימת התחברות גם למאמן וגם למתאמן תקינה למערכת.
3. למאמן קיים לפחות מתאמן אחד שמחובר אליו.

שלבים :

1. להתחבר לאפליקציה כמאמן.
2. להיכנס למסך קביעת אימון ב + .
3. לבחור שם של קבוצת מתאמנים.
4. לבחור תאריך לאימון.
5. לבחור שעה לאימון.
6. להזין מיקום לאימון/ בדיקה שזה עובד גם עם מיקום על המפה.
7. לבחור מתאמן מהרשימה.
8. ללחוץ על קבע אימון.
9. לצאת מחשבון המאמן ולעבור לחשבון המתאמן .
10. להתחבר למתאמן שנבחר בשלב 3.
11. להיכנס לאימונים המתוכננים.

תוצאה צפויה:

1. בחשבון המתאמן שנבחר, מופיע אימון חדש במסך האימונים המתוכננים.
2. פרטי האימון המוצגים אצל המתאמן תואמים למה שנקבע ע"י המאמן: תאריך, שעה, מיקום.
3. אין כפילות של אימון ואין הודעת שגיאה.

תרחיש בדיקה 2: שליחת הודעה ממאמן למתאמן וקבלת התראה

שם התרחיש:

שליחת הודעה ממאמן למתאמן והצגת התראה על הודעה חדשה

תנאי קדם:

1. האפליקציה מותקנת ופועלת.
2. המאמן מחובר לאפליקציה עם חשבון תקין.
3. למאמן קיימים מתאמנים מחוברים לחשבון.
4. המתאמן מחובר לחשבון תקין באפליקציה.

שלבים:

1. להתחבר לאפליקציה מאמן.
2. להיכנס לשליחת הודעה למתאמנים.
3. להקליד תוכן הודעה בשדה הטקסט.
4. לבחור מתאמן אחד או יותר מרשימת המתאמנים (הרשימה מציגה את כל המתאמנים של המאמן).
5. ללחוץ על כפתור שליחה.
6. להתנתק מחשבון המאמן ולעבור לחשבון המתאמן שנבחר.
7. להתחבר לאפליקציה למתאמן.
8. לבדוק את סמל הפעמון (Notifications) בחלק העליון של המסך.
9. ללחוץ על סמל הפעמון כדי לצפות בהתראה ובהודעה.

תוצאה צפויה:

1. לאחר שליחת ההודעה, הפעולה מתבצעת ללא שגיאה.
2. בחשבון המתאמן מופיעה נקודה אדומה על סמל הפעמון, המעידה על הודעה חדשה.
3. בלחיצה על סמל הפעמון נפתח מסך ההתראות.
4. ההודעה שנשלחה ע"י המאמן מוצגת למתאמן עם התוכן שנשלח.
5. לאחר צפייה בהודעה, סימון ההתראה החדשה מתעדכן בהתאם (מוריד את הנקודה האדומה)

תרחיש בדיקה 3: הוספת מטבעות למתאמן על ידי מאמן

שם התרחיש:

הוספת מטבעות למתאמן מחשבון המאמן והצגתם אצל המתאמן

תנאי קדם:

1. האפליקציה מותקנת ופועלת.
2. המאמן מחובר לאפליקציה עם חשבון תקין.
3. למאמן קיימים מתאמנים מחוברים לחשבון.
4. המתאמן מחובר לחשבון תקין באפליקציה.

שלבים:

1. להתחבר לאפליקציה כמאמן.
2. להיכנס למסך רשימת מתאמנים.
3. לבחור מתאמן מהרשימה.
4. ללחוץ על פעולה של הוספת מטבעות.
5. לאשר את הפעולה (לחיצה על כפתור אישור).
6. לצאת מחשבון המאמן או לעבור לחשבון המתאמן שנבחר.

7. להתחבר לאפליקציה כמתאמן.
8. להיכנס למסך הראשי או למסך הפרופיל שבו מוצג מספר המטבעות.

תוצאה צפויה:

1. פעולת הוספת המטבעות מתבצעת ללא שגיאה.
2. אצל המתאמן מוצג עדכון של כמות המטבעות.
3. מספר המטבעות של המתאמן גדל ב-50 מטבעות.
4. אין כפילות או חישוב שגוי של כמות המטבעות.

חלק B: בדיקות יחידה

```
StepBuyStepUnitTest.java x build.gradle.kts (Step Buy Step) build.gradle.kts (:app) BroadcastMessage
1 package com.example.stepbustep;
2 import static org.junit.Assert.*;
3 import static org.mockito.Mockito.*;
4 import com.example.stepbustep.ActivityTrainee.TraineeStoreScreen.ShoeProgressionManager;
5 import com.example.stepbustep.model.Equipment;
6 import com.example.stepbustep.model.Message;
7 import org.junit.Test;
8
9 public class StepBuyStepUnitTest {
10     @Test
11     public void testPurchaseWithInsufficientFunds() {
12         // Current shoe level of the trainee
13         int currentLevel = 1;
14         // Trainee's coin balance (not enough to buy next level)
15         long lowBalance = 500;
16         // Call the method under test
17         boolean canBuy = ShoeProgressionManager.canPurchaseNextLevel(currentLevel, lowBalance);
18         // Assert that the purchase is NOT allowed
19         assertFalse(
20             message: "Trainee should not be able to buy shoes with insufficient coins",
21             canBuy
22         );
23     }
24     @Test
25     public void testEquipmentModelConsistency() {
26         // Expected values for the Equipment object
27         String expectedId = "eq_1";
28         String expectedName = "Speed Shoes";
29         String expectedType = "running_shoes";
30         int expectedTier = 2;
31         int expectedPrice = 3000;
32         double expectedMultiplier = 1.25;
33         // Create a new Equipment object
34         Equipment equipment = new Equipment(
35             expectedId,
36             expectedName,
37             expectedType,
38             expectedTier,
39             expectedPrice,
40             expectedMultiplier
41         );
42         // Verify that each getter returns the correct value
43         assertEquals(expectedId, equipment.getId());
44         assertEquals(expectedName, equipment.getName());
45         assertEquals(expectedType, equipment.getType());
46         assertEquals(expectedTier, equipment.getTier());
47         assertEquals(expectedPrice, equipment.getPrice());
48         // For double comparison, include a delta value
49         assertEquals(expectedMultiplier, equipment.getMultiplier(), delta: 0.0001);
50     }
51     @Test
52     public void testMessageWithMock() {
53         // Create a mock instance of the Message class
54         Message mockedMessage = mock(Message.class);
55         when(mockedMessage.getCoachId()).thenReturn(value: "Coach_123");
56         when(mockedMessage.getMessageText()).thenReturn(value: "Keep Running!");
57         // Call the mocked methods
58         String coachId = mockedMessage.getCoachId();
59         String text = mockedMessage.getMessageText();
60         // Assert that the returned values match the mocked behavior
61         assertEquals(expected: "Coach_123", coachId);
62         assertEquals(expected: "Keep Running!", text);
63         // Verify that the getters were called exactly once
64         verify(mockedMessage, times(wantedNumberOfInvocations: 1)).getCoachId();
65         verify(mockedMessage, times(wantedNumberOfInvocations: 1)).getMessageText();
66         // Verify that no other interactions happened with the mock
67         verifyNoMoreInteractions(mockedMessage);
68     }
69 }
70 }
```

✓ 3 tests passed 3 tests total, 2 sec 229 ms

Executing tasks: [:app:testDebugUnitTest, --tests, com.example.stepbustep.StepBuyStepUnitTest] in project C:\Us

חלק C: בדיקות UI

```
1 package com.example.stepbustep;
2 import static androidx.test.espresso.Espresso.onView;
3 import static androidx.test.espresso.action.ViewActions.*;
4 import static androidx.test.espresso.assertion.ViewAssertions.matches;
5 import static androidx.test.espresso.matcher.ViewMatchers.*;
6 import static org.hamcrest.Matchers.not;
7 import androidx.test.ext.junit.rules.ActivityScenarioRule;
8 import androidx.test.ext.junit.runners.AndroidJUnit4;
9 import com.example.stepbustep.ActivityCoach.CoachHomeScreen.CoachHomeActivity;
10 import org.junit.Rule;
11 import org.junit.Test;
12 import org.junit.runner.RunWith;
13
14 @RunWith(AndroidJUnit4.class)
15 public class BroadcastMessageDialogUiTest {
16     // Rule that launches CoachHomeActivity before each test
17     @Rule
18     public ActivityScenarioRule<CoachHomeActivity> activityRule = new ActivityScenarioRule<>(CoachHome
19     // Test that opening the broadcast dialog shows the message input field
20     @Test
21     public void testOpenBroadcastDialog_showsMessageInput() {
22         // Click the broadcast message button on CoachHome screen
23         onView(withId(R.id.btnBroadcastMessage)).perform(click());
24         // Verify that the message input field is displayed
25         onView(withId(R.id.etMessage)).check(matches(isDisplayed()));
26     }
27     // Test default state: "Send to all trainees" is checked and list is hidden
28
29     @Test
30     public void testDefaultState_sendToAllChecked_listHidden() {
31         // Open the broadcast message dialog
32         onView(withId(R.id.btnBroadcastMessage)).perform(click());
33         // Verify that the checkbox is checked by default
34         onView(withId(R.id.cbSendToAll)).check(matches(isChecked()));
35         // Verify that the trainees list is hidden
36         onView(withId(R.id.rvTraineesList)).check(matches(withEffectiveVisibility(Visibility.GONE)));
37         // Verify that the selection text is also hidden
38         onView(withId(R.id.tvSelectTrainees)).check(matches(withEffectiveVisibility(Visibility.GONE)));
39     }
40     // Test that unchecking "Send to all" shows the trainees selection UI
41     @Test
42     public void testUncheckSendToAll_showsTraineesSelection() {
43         // Open the broadcast message dialog
44         onView(withId(R.id.btnBroadcastMessage)).perform(click());
45         // Uncheck the "Send to all trainees" checkbox
46         onView(withId(R.id.cbSendToAll)).perform(click());
47         // Verify that the selection text becomes visible
48         onView(withId(R.id.tvSelectTrainees)).check(matches(withEffectiveVisibility(Visibility.VISIBLE)));
49         // Verify that the trainees RecyclerView becomes visible
50         onView(withId(R.id.rvTraineesList)).check(matches(withEffectiveVisibility(Visibility.VISIBLE)));
51     }
52     // Test validation: sending an empty message is not allowed
53
54     @Test
55     public void testSendEmptyMessage_keepsInputEmpty() {
56         // Open the broadcast message dialog
57         onView(withId(R.id.btnBroadcastMessage)).perform(click());
58         // Ensure the message input is empty
59         onView(withId(R.id.etMessage)).perform(replaceText(stringToBeSet: ""), closeSoftKeyboard());
60         // Click the send button
61         onView(withId(R.id.btnSend)).perform(click());
62         // Verify that the message input is still empty
63         onView(withId(R.id.etMessage)).check(matches(withText("")));
64     }
65 }
```

✓ Test Results	15 s	4/4
✓ BroadcastMessageDialogUITest	15 s	4/4
✓ testOpenBroadcastDialog_showsMessageInput	7 s	✓
✓ testDefaultState_sendToAllChecked_listHidden	2 s	✓
✓ testUncheckSendToAll_showsTraineesSelector	2 s	✓
✓ testSendEmptyMessage_keepsInputEmpty	3 s	✓

חלק 4 : Code Review

שם המודול: MyMoveFragment
סוג: Android Fragment באפליקציית
ת"ז: 323834309

בעיות שנמצאו בקוד

1. ערבוב לוגיקת ממשק משתמש ולוגיקה עסקית

ה-`Fragment` מכיל לוגיקה עסקית כגון זיהוי האם המשתמש הוא הלקוח או השותף, קבלת החלטות לגבי ביטול הובלה לפי תאריך, וטיפול בסטטוס ההובלה.

המלצה: להעביר לוגיקה עסקית ל-`ViewModel` ולשמור ב-`Fragment` רק עדכון `UI`.

2. שימוש ישיר ב-`Repository` מתוך ה-`Fragment`

במודול מתבצעת יצירה ישירה של מחלקות `Repository` מתוך ה-`Fragment`. דבר זה יוצר תלות חזקה בין שכבת ה-`UI` לשכבת הנתונים, מקשה על בדיקות `Unit` ופוגע במבנה הארכיטקטוני של האפליקציה.

המלצה: לבצע גישה לנתונים דרך `ViewModel / Injection` ולא ליצור `Repository` מתוך `UI`.

3. הגדרת מאזיני לחיצה בתוך `Observers` של `LiveData`

מאזיני הלחיצה לכפתורי אישור ודחייה מוגדרים בתוך `observe` של `LiveData`. במקרה של עדכון חוזר של הנתונים, פעולה זו עלולה לגרום להתנהגות לא צפויה ולפגוע בקריאות הקוד.

שיפור מומלץ: להגדיר מאזינים פעם אחת בלבד, ולהשתמש בערך העדכני מה-`ViewModel` בזמן הלחיצה.

דוגמה לשיפור

העברת מאזיני הלחיצה להגדרה חד-פעמית, והשארת ה-`Observer` אחראי רק לעדכון ממשק המשתמש.

```
btnApproveReq.setOnClickListener(v -> {  
    MoveRequest req = viewModel.getIncomingRequest().getValue();  
    if (req != null) viewModel.approveMatch(req);  
});  
  
btnRejectReq.setOnClickListener(v -> {  
    MoveRequest req = viewModel.getIncomingRequest().getValue();  
    if (req != null) viewModel.rejectMatch(req);  
});
```

שיפור זה מעלה את קריאות הקוד, מונע ריבוי חיבורים למאזינים, ומשפר את מבנה המודול.

בעיות שנמצאו בקוד

1) Magic Strings וסטטוסים לא אחידים

הקוד משתמש בהרבה ערכי טקסט קשיחים כגון:
"OPEN", "CONFIRMED", "CANCELED", "COMPLETED", "waiting_for_mover",
"approved"
הדבר מגדיל סיכון לטעויות כתיב, חוסר עקביות ושכירה שקטה של לוגיקה.

זה גורם לבאגים שקשה לאתר + תחזוקה קשה יותר.
המלצה - להגדיר קבועים/Enum לכל הסטטוסים וה-fields המרכזיים.

2) פעולות כתיבה מרובות ללא Transaction (סיכון לחוסר עקביות)

בחלק מהפעולות יש עדכונים מרובים על מסמכים שונים (Move + Chat + User + Request).
בחלק מהמקרים משתמשים ב-Batch/Transaction (טוב), אבל במקרים אחרים יש update רגיל שיכול להצליח חלקית אם משהו נכשל באמצע.

דוגמה:

approveMatchByPartner עושה קודם get ואז update במסמך אחר ללא Transaction.

זה גורם למצב שבו חלק מהנתונים עודכנו וחלק לא נתונים לא עקביים.
המלצה: במקרים של "תלות בין מסמכים" להשתמש ב-Batch או Transaction אחד.

3) טיפול בשגיאות ו-Null Safety חלקי

יש מקומות שבהם מתבצעות פעולות על task.getResult בלי בדיקה מספקת, לדוגמה:

```
.continueWith(task -> !task.getResult().isEmpty());
```

אם task נכשל או getResult הוא null - זה יכול לקרוס.

זה גורם קריסות בזמן ריצה (Runtime crashes). המלצה: להוסיף בדיקות task.isSuccessful()
ולטפל ב-Exceptions בצורה מסודרת.

דוגמה לשיפור

שיפור פונקציה שמחזירה האם יש הובלה מאושרת פעילה (hasActiveConfirmedMove) כדי למנוע קריסה במקרה שהשאלתה נכשלת.

תיקון:

```
public Task<Boolean> hasActiveConfirmedMove(String customerId) {  
    return db.collection(COLLECTION_MOVES)  
        .whereEqualTo("customerId", customerId)  
        .whereEqualTo("status", "CONFIRMED")  
        .limit(1)  
        .get()  
        .continueWith(task -> {  
            if (!task.isSuccessful() || task.getResult() ==  
null) {  
                throw task.getException() != null ?  
task.getException()  
                : new Exception("שגיאה בשליפת הובלות מאושרות");  
            }  
            return !task.getResult().isEmpty();  
        });  
}
```

- מונע NullPointerException
- הופך את ההתנהגות לצפויה במקרה של תקלה
- משפר יציבות ואמינות מערכת

שם המודול: ChatActivity

סוג: Activity (Android)

ת"ז: 207304338

בעיות עיקריות שנמצאו בקוד

1) תלות ישירה ב-Repository בתוך ה-Activity

למרות שימוש ב-ViewModel, עדיין קיימת קריאה ישירה:
`new UserRepository().getUserNameById`
זה יוצר תלות ב-UI לשכבת הדאטה ומקשה על בדיקות ועל תחזוקה.

המלצה: להעביר את טעינת שם המשתמש ל-ChatViewModel (או להעביר את השם ב-Intent / לשמור בקאש).

2) מצב שבו נשלחת הודעה בלי שם משתמש טעון

currentUserName נטען אסינכרונית, אבל `sendMessage()` יכול להיקרא לפני שהשם הגיע הודעות עלולות להישלח עם `null`/ריק.

המלצה:

- להשהות שליחה עד שיש `currentUserName`
- להשתמש בשם ברירת מחדל
- להוציא שם משתמש מתוך `cache / auth`.

3) לוגיקת UI מרובה בתוך Activity

הפונקציה `updateConfirmCardUi` מכילה הרבה תנאים שמערבבים לוגיקה ותצוגה. זה עובד, אבל קשה להרחיב/לבדוק.

המלצה: להעביר את הסטייט ל-ViewModel כמודל UI (למשל `ConfirmCardState`) ואז ה-Activity רק "מצייר".

דוגמה לשיפור

מניעת שליחת הודעה לפני טעינת שם המשתמש, כדי לא לשלוח הודעות עם `null`.

```
private void sendMessage() {  
    String text = editInput.getText().toString().trim();  
    if (TextUtils.isEmpty(text)) return;
```

```
        if (currentUserName == null || currentUserName.trim().isEmpty())
        {
            Toast.makeText(this, "טוען פרטי משתמש...נסה שוב בעוד רגע",
Toast.LENGTH_SHORT).show();
            return;
        }

        editInput.setText("");
        chatViewModel.sendMessage(chatId, text, currentUserId,
currentUserName);
    }
```